Sales and Customer Analytics System

Aim:

To analyze sales data in the EcoCart database using SQL aggregate functions and generate reports about customer purchases, product performance, and category-wise sales.

Objectives:

- To calculate total sales, average sales, and order amounts.
- To analyze customer purchasing behavior.
- To identify top customers based on purchase amount.
- To identify best-selling and least-selling products.
- To perform category-wise sales analysis.
- To generate business analytics reports for decision-making.

Apply SQL Aggregate Functions:

Aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX() are used to summarize sales data. These functions help calculate the total number of orders, total revenue, average order value, and highest and lowest order amounts.

1.Generate Total Sales Reports:

Tasks: Write SQL queries to calculate:

1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS total_orders FROM orders;

TOTAL_ORDERS
------------
          10
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(total_amount) AS total_revenue FROM orders;

TOTAL_REVENUE
-------------
         4300
```

3. Average order value.

```
SQL> SELECT AVG(total_amount) AS average_order_value FROM orders;

AVERAGE_ORDER_VALUE
-------------------
                430
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(total_amount) AS highest_order, MIN(total_amount) AS lowest_order FROM orders;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
          850          180
```

5. Total sales for a specific period.

```
SQL> SELECT SUM(total_amount) AS period_sales FROM orders WHERE order_date >= DATE '2026-09-01' AND order_date < DATE '2026-10-01';

PERIOD_SALES
------------
        4300
```

2. Customer Purchase Analysis:

Customer purchase analysis is used to understand customer buying behavior.

The NVL() function replaces NULL values with zero for customers who have not placed any orders.

1. Total number of orders placed by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c
.customer_id, c.customer_name;

CUSTOMER_I CUSTOMER_NAME        TOTAL_ORDERS
---------- -------------------- ------------
CUS010     Lavanya M                       1
CUS004     Keerthana S                     1
CUS008     Nithya R                        1
CUS002     Harini Devi                     1
CUS001     Sanjay Kumar                    1
CUS003     Praveen Raj                     1
CUS007     Ashwin Anand                    1
CUS009     Surya Prakash                   1
CUS006     Pavithra M                      1
CUS005     Dinesh Kumar                    1

10 rows selected.
```

2. Total amount spent by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(SUM(o.total_amount), 0) AS total_spent FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id G
ROUP BY c.customer_id, c.customer_name;

CUSTOMER_I CUSTOMER_NAME        TOTAL_SPENT
---------- -------------------- -----------
CUS010     Lavanya M                    520
CUS004     Keerthana S                  400
CUS008     Nithya R                     360
CUS002     Harini Devi                  250
CUS001     Sanjay Kumar                 520
CUS003     Praveen Raj                  600
CUS007     Ashwin Anand                 420
CUS009     Surya Prakash                200
CUS006     Pavithra M                   850
CUS005     Dinesh Kumar                 180

10 rows selected.
```

3. Average spending per customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(AVG(o.total_amount), 0) AS average_spending FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer
_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_I CUSTOMER_NAME        AVERAGE_SPENDING
---------- -------------------- ----------------
CUS010     Lavanya M                         520
CUS004     Keerthana S                       400
CUS008     Nithya R                          360
CUS002     Harini Devi                       250
CUS001     Sanjay Kumar                      520
CUS003     Praveen Raj                       600
CUS007     Ashwin Anand                      420
CUS009     Surya Prakash                     200
CUS006     Pavithra M                        850
CUS005     Dinesh Kumar                      180

10 rows selected.
```

4. Customers with maximum purchase amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM orders GROUP BY cus
tomer_id));

CUSTOMER_I CUSTOMER_NAME        TOTAL_SPENT
---------- -------------------- -----------
CUS006     Pavithra M                   850
```

5. Customers with fewer purchases.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c
.customer_id, c.customer_name HAVING COUNT(o.order_id) <= 1;

CUSTOMER_I CUSTOMER_NAME        TOTAL_ORDERS
---------- -------------------- ------------
CUS010     Lavanya M                       1
CUS004     Keerthana S                     1
CUS008     Nithya R                        1
CUS002     Harini Devi                     1
CUS001     Sanjay Kumar                    1
CUS003     Praveen Raj                     1
CUS007     Ashwin Anand                    1
CUS009     Surya Prakash                   1
CUS006     Pavithra M                      1
CUS005     Dinesh Kumar                    1

10 rows selected.
```

3. Top Customers Based on Purchase Amount:

This analysis identifies the top five customers based on their total spending. It also helps find customers who place the maximum number of orders, frequent customers, and high-value customers. These details help the company improve customer satisfaction and loyalty.

1. Top 5 customers based on total spending.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id
 GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_I CUSTOMER_NAME        TOTAL_SPENT
---------- -------------------- -----------
CUS006     Pavithra M                   850
CUS003     Praveen Raj                  600
CUS010     Lavanya M                    520
CUS001     Sanjay Kumar                 520
CUS007     Ashwin Anand                 420
```

2. Customers with maximum number of orders.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cust
omer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY customer_id));

CUSTOMER_I CUSTOMER_NAME        TOTAL_ORDERS
---------- -------------------- ------------
CUS010     Lavanya M                       1
CUS004     Keerthana S                     1
CUS008     Nithya R                        1
CUS002     Harini Devi                     1
CUS001     Sanjay Kumar                    1
CUS003     Praveen Raj                     1
CUS007     Ashwin Anand                    1
CUS009     Surya Prakash                   1
CUS006     Pavithra M                      1
CUS005     Dinesh Kumar                    1

10 rows selected.
```

3. Frequent customers.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cust
omer_id, c.customer_name HAVING COUNT(o.order_id) >= 2;

no rows selected
```

4. High-value customers.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) >= 50000 ORDER BY total_spent DESC;

no rows selected
```

4. Identify Best-Selling Products:

Product analysis identifies products with the highest sales quantity and revenue. It also finds least-selling products and products that may require promotional offers. This information helps the company manage inventory and improve product sales.

1. Products with maximum sales quantity.

```
SQL> SELECT p.product_id, p.product_name, SUM(od.quantity) AS quantity_sold FROM product p JOIN order_details od ON p.product_id = od.product_id GROUP BY p.
product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quantity) AS total_quantity FROM order_details GROUP BY pr
oduct_id));

PRODUCT_ID   PRODUCT_NAME              QUANTITY_SOLD
------------ ------------------------- -------------
PRO008       Cotton Tote Bag                       3
PRO007       Eco Diary                             3
```

2. Products generating highest revenue.

```
SQL> SELECT p.product_id, p.product_name, SUM(od.quantity * od.price) AS revenue FROM product p JOIN order_details od ON p.product_id = od.product_id GROUP
BY p.product_id, p.product_name ORDER BY revenue DESC;

PRODUCT_ID   PRODUCT_NAME                REVENUE
---------- ------------------------- ----------
PRO009       Eco Cotton Kurta               850
PRO002       Natural Honey                  640
PRO006       Steel Lunch Box                600
PRO008       Cotton Tote Bag                540
PRO007       Eco Diary                      420
PRO005       Bamboo Tray                    400
PRO003       Aloe Face Cream                250
PRO001       Organic Oats                   200

8 rows selected.
```

3. Least-selling products.

```
SQL> SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.product_i
d GROUP BY p.product_id, p.product_name ORDER BY quantity_sold ASC;

PRODUCT_ID   PRODUCT_NAME              QUANTITY_SOLD
---------- ------------------------- -------------
PRO004       Bamboo Comb                         0
PRO010       Organic Seed Pack                   0
PRO006       Steel Lunch Box                     1
PRO001       Organic Oats                        1
PRO003       Aloe Face Cream                     1
PRO005       Bamboo Tray                         1
PRO009       Eco Cotton Kurta                    1
PRO002       Natural Honey                       2
PRO007       Eco Diary                           3
PRO008       Cotton Tote Bag                     3

10 rows selected.
```

4. Products requiring promotion.

```
SQL>  SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.product_
id GROUP BY p.product_id, p.product_name HAVING NVL(SUM(od.quantity), 0) <= 1;

PRODUCT_ID   PRODUCT_NAME              QUANTITY_SOLD
---------- ------------------------- -------------
PRO005       Bamboo Tray                         1
PRO003       Aloe Face Cream                     1
PRO006       Steel Lunch Box                     1
PRO009       Eco Cotton Kurta                    1
PRO001       Organic Oats                        1
PRO004       Bamboo Comb                         0
PRO010       Organic Seed Pack                   0

7 rows selected.
```

5. Category-wise Sales Analysis:

Category-wise sales analysis calculates total sales and the number of product units sold in each category. It identifies the category generating the highest revenue and calculates the average sales value. This helps the company understand which categories perform well.

1. Total sales generated by each category.

```
SQL> SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_sales FROM category c LEFT JOIN product p ON c.category_id = p.cate
gory_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGORY_ID  CATEGORY_NAME        TOTAL_SALES
---------- -------------------- -----------
CAT004       Kitchen Items               600
CAT005       Eco Stationery              420
CAT002       Personal Care               250
CAT006       Cloth Bags                  540
CAT010       Cleaning Products             0
CAT008       Sustainable Fashion         850
CAT007       Home Decor                    0
CAT009       Farming Products              0
CAT001       Organic Food                840
CAT003       Bamboo Products             400

10 rows selected.
```

2. Number of products sold in each category.

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JOIN order_
details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME        PRODUCTS_SOLD
-------------------- -------------
Sustainable Fashion              1
Organic Food                     3
Bamboo Products                  1
Kitchen Items                    1
Personal Care                    1
Cloth Bags                       3
Eco Stationery                   3
Farming Products                 0
Cleaning Products                0
Home Decor                       0

10 rows selected.
```

3. Highest revenue-generating category.

```
SQL> SELECT * FROM (SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p ON c.cat
egory_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY total_revenue DESC) WHE
RE ROWNUM = 1;

CATEGORY_ID  CATEGORY_NAME        TOTAL_REVENUE
------------ -------------------- -------------
CAT008       Sustainable Fashion            850
```

4. Average sales per category.

```
SQL> SELECT c.category_name, NVL(AVG(od.quantity * od.price), 0) AS average_sales FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT
JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME        AVERAGE_SALES
-------------------- -------------
Sustainable Fashion            850
Organic Food                   280
Bamboo Products                400
Kitchen Items                  600
Personal Care                  250
Cloth Bags                     270
Eco Stationery                 420
Farming Products                 0
Cleaning Products                0
Home Decor                       0

10 rows selected.
```

6. Business Analytics Reports:

Four business reports can be generated from the EcoCart database:

Report 1: Sales Performance Report

- Displays total sales, number of orders, average order value, and highest order value.

```
SQL> SELECT COUNT(*) AS number_of_orders, SUM(total_amount) AS total_sales, AVG(total_amount) AS average_order_value, MAX(total_amount) AS highest_order_val
ue FROM orders;

NUMBER_OF_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
---------------- ----------- ------------------- -------------------
              10        4300                 430                 850
```

Report 2: Customer Analytics Report

- Displays customer names, number of orders, and total spending.

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS number_of_orders, NVL(SUM(o.total_amount), 0) AS total_spending FROM customer c LEFT JOIN orders o ON c.cu
stomer_id = o.customer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_SPENDING
-------------------- ---------------- --------------
Lavanya M                           1            520
Keerthana S                         1            400
Nithya R                            1            360
Harini Devi                         1            250
Sanjay Kumar                        1            520
Praveen Raj                         1            600
Ashwin Anand                        1            420
Surya Prakash                       1            200
Pavithra M                          1            850
Dinesh Kumar                        1            180

10 rows selected.
```

Report 3: Product Performance Report

- Displays product names, quantity sold, and revenue generated.

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS revenue_generated FROM product p LEFT JOIN ord
er_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name ORDER BY revenue_generated DESC;

PRODUCT_NAME              QUANTITY_SOLD REVENUE_GENERATED
------------------------- ------------- -----------------
Eco Cotton Kurta                      1               850
Natural Honey                         2               640
Steel Lunch Box                       1               600
Cotton Tote Bag                       3               540
Eco Diary                             3               420
Bamboo Tray                           1               400
Aloe Face Cream                       1               250
Organic Oats                          1               200
Bamboo Comb                           0                 0
Organic Seed Pack                     0                 0

10 rows selected.
```

Report 4: Category Analysis Report

- Displays category names, total product units sold, and total revenue.

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS total_products_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN
 product p ON c.category_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY tota
l_revenue DESC;

CATEGORY_NAME        TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
-------------------- ------------------- -------------
Sustainable Fashion                    1           850
Organic Food                           3           840
Kitchen Items                          1           600
Cloth Bags                             3           540
Eco Stationery                         3           420
Bamboo Products                        1           400
Personal Care                          1           250
Farming Products                       0             0
Home Decor                             0             0
Cleaning Products                      0             0

10 rows selected.
```